

C04-AT1

Design and Analysis of Algorithm

Name: A.Bhavya(192424417)

Q3. Traveling Salesman Problem (TSP)

A delivery agent must visit multiple cities exactly once and return to the starting location. The travel costs between each pair of cities are given in a matrix. The goal is to determine the most cost-efficient route that minimizes total travel cost. Use Dynamic Programming (Held-Karp method) to solve the problem efficiently.

Sample Input:

n = 4

Cost =

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Expected Output:

Minimum Cost = 80

Introduction

The travel cost between every pair of cities is represented using a cost matrix. The main objective is to find a route that visits every city exactly once and has the minimum total travel cost. For a small number of cities, TSP can be solved using brute force by checking all possible routes. However, the number of possible routes increases rapidly as the number of cities increases. Therefore, Dynamic Programming can be used to reduce repeated calculations.

The Held-Karp algorithm is a Dynamic Programming technique used to solve TSP more efficiently. It stores the results of previously solved subproblems and uses them to construct the optimal solution.

Objectives

The main objectives of solving the TSP using Dynamic Programming are:

1. To find the minimum-cost route for visiting all cities.

2. To visit each city exactly once.
3. To return to the starting city.
4. To implement the Held-Karp Dynamic Programming technique.
5. To analyze the time and space complexity of the algorithm.
6. Given Cost Matrix
The given cost matrix is:

City	1	2	3	4
1	0	10	15	20
2	10	0	35	25
3	15	35	0	30
4	20	25	30	0

For example:

- City 1 $\rightarrow$ City 2 = 10
- City 1 $\rightarrow$ City 3 = 15
- City 1 $\rightarrow$ City 4 = 20
- City 2 $\rightarrow$ City 4 = 25
- City 4 $\rightarrow$ City 3 = 30
- City 3 $\rightarrow$ City 1 = 15

5. Dynamic Programming Approach

The Held-Karp algorithm uses subsets of cities to solve TSP.

Let:

$DP[S][i]$

represent the minimum cost required to:

- start from City 1,
- visit all cities in the subset S,

- and finish at City i.

The recurrence relation is:

$$DP[S][i] = \min(DP[S - \{i\}][j] + \text{Cost}[j][i])$$

where j represents a previously visited city.

The initial states are:

$$DP[\{1,2\}][2] = 10$$

$$DP[\{1,3\}][3] = 15$$

$$DP[\{1,4\}][4] = 20$$

6. Step-by-Step Solution

Step 1: Starting from City 1

The possible first movements are:

$$1 \rightarrow 2 = 10$$

$$1 \rightarrow 3 = 15$$

$$1 \rightarrow 4 = 20$$

Step 2: Visiting Three Cities

For cities {1, 2, 3}:

$$1 \rightarrow 3 \rightarrow 2 = 15 + 35 = 50$$

$$1 \rightarrow 2 \rightarrow 3 = 10 + 35 = 45$$

For cities {1, 2, 4}:

$$1 \rightarrow 4 \rightarrow 2 = 20 + 25 = 45$$

$$1 \rightarrow 2 \rightarrow 4 = 10 + 25 = 35$$

For cities {1, 3, 4}:

$$1 \rightarrow 4 \rightarrow 3 = 20 + 30 = 50$$

$$1 \rightarrow 3 \rightarrow 4 = 15 + 30 = 45$$

Step 3: Visit All Cities

Consider the route:

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$$

Total cost:

$$= \text{Cost}(1,2) + \text{Cost}(2,4) + \text{Cost}(4,3) + \text{Cost}(3,1)$$

$$= 10 + 25 + 30 + 15$$

$$= 80$$

Another route is:

$$1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

Its cost is:

$$15 + 30 + 25 + 10 = 80$$

Therefore:

Minimum Cost = 80

Algorithm

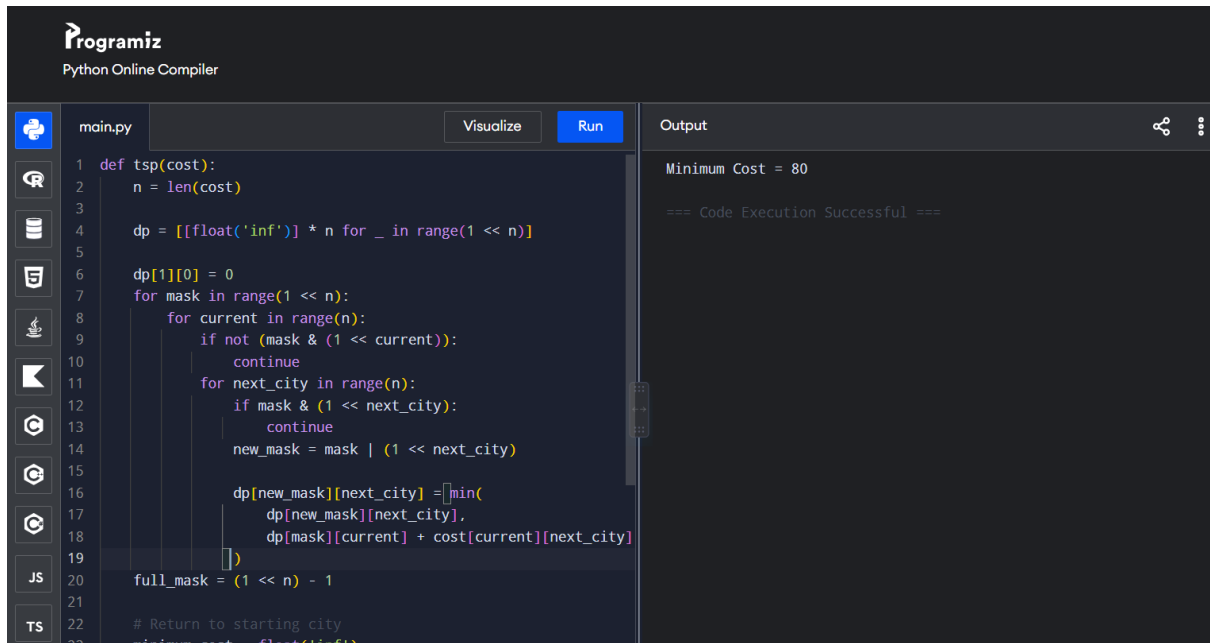
1. Start from City 1.
2. Represent visited cities using a binary mask.
3. Initialize the DP table with infinity.
4. Set the starting city cost to 0.
5. For every subset of visited cities, consider each possible current city.
6. Try visiting every unvisited city.
7. Update the DP table with the minimum cost.
8. After visiting all cities, add the cost of returning to City 1.
9. Select the minimum total cost.
10. Display the minimum cost.

Code

```
main.py Visualize Run
1 def tsp(cost):
2     n = len(cost)
3
4     dp = [[float('inf')] * n for _ in range(1 << n)]
5
6     dp[1][0] = 0
7     for mask in range(1 << n):
8         for current in range(n):
9             if not (mask & (1 << current)):
10                continue
11            for next_city in range(n):
12                if mask & (1 << next_city):
13                    continue
14                new_mask = mask | (1 << next_city)
15
16                dp[new_mask][next_city] = min(
17                    dp[new_mask][next_city],
18                    dp[mask][current] + cost[current][next_city]
19                )
20
21     full_mask = (1 << n) - 1
22
23     # Return to starting city
24     minimum_cost = float('inf')
```

```
main.py Visualize Run
1 def tsp(cost):
2
35
36     # Return to starting city
37     minimum_cost = float('inf')
38
39     for city in range(1, n):
40         minimum_cost = min(
41             minimum_cost,
42             dp[full_mask][city] + cost[city][0]
43         )
44
45     return minimum_cost
46
47
48 # Cost matrix
49 cost = [
50     [0, 10, 15, 20],
51     [10, 0, 35, 25],
52     [15, 35, 0, 30],
53     [20, 25, 30, 0]
54 ]
55
```

Output



The screenshot shows the Programiz Python Online Compiler interface. The main editor displays a Python script for solving the Traveling Salesman Problem (TSP) using dynamic programming. The script defines a function `tsp(cost)` that takes a list of city costs and returns the minimum cost. The output panel on the right shows the result: "Minimum Cost = 80" and "=== Code Execution Successful ===".

```
1 def tsp(cost):
2     n = len(cost)
3
4     dp = [[float('inf')] * n for _ in range(1 << n)]
5
6     dp[1][0] = 0
7     for mask in range(1 << n):
8         for current in range(n):
9             if not (mask & (1 << current)):
10                continue
11            for next_city in range(n):
12                if mask & (1 << next_city):
13                    continue
14                new_mask = mask | (1 << next_city)
15
16                dp[new_mask][next_city] = min(
17                    dp[new_mask][next_city],
18                    dp[mask][current] + cost[current][next_city]
19                )
20
21     full_mask = (1 << n) - 1
22
23     # Return to starting city
24     minimum_cost = float('inf')
```

Output: Minimum Cost = 80
=== Code Execution Successful ===

Time Complexity

$O(n^2 \times 2^n)$

Space Complexity

$O(n \times 2^n)$

Advantages

- Avoids repeated calculations.
- Provides the optimal solution.
- More efficient than brute-force for increasing numbers of cities.
- Uses previously calculated subproblems.
- Suitable for route and transportation optimization.

Applications

TSP has several real-world applications:

- Delivery route optimization.
- Transportation planning.

- Logistics and supply-chain management.
- Vehicle route planning.
- Robot path planning.
- Network optimization.
- Tour and travel planning.

Conclusion

The Traveling Salesman Problem demonstrates how Dynamic Programming can be used to solve an optimization problem efficiently. The Held-Karp method divides the problem into smaller subproblems based on visited cities and stores their results to avoid repeated computation.

For the given four-city cost matrix, the algorithm finds the optimal route $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$ with a minimum travel cost of 80. Thus, Dynamic Programming provides an effective approach for obtaining the optimal solution to TSP.